

WA3851 Lab Guide

# Flexbox, Bootstrap, and Responsive Design for Leaders



Part of  
**Accenture**

© 2026 Ascendient, LLC

Revision 1.0.0 published on 2026-05-22

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

**Trademark Notice:** Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: [getinfousa@ascendientlearning.com](mailto:getinfousa@ascendientlearning.com)

Canada: 1-877-812-8887 toll free, email: [getinfo@ascendientlearning.com](mailto:getinfo@ascendientlearning.com)

## Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

### ● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

### ● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

### 🚨 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

# Table of Contents

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 1. Evergreen Insurance Quote Page Responsive Layout with Flexbox..... | 5  |
| 1.1. A Note About Keyboard Shortcuts.....                             | 6  |
| 1.2. Why You're Doing This Lab .....                                  | 6  |
| 1.3. What's Already Set Up on Your Lab VM.....                        | 7  |
| 1.4. Open the Starter in VS Code .....                                | 7  |
| 1.5. Launch the Page with Live Server.....                            | 8  |
| 1.6. Observe the "Before" State in DevTools Device Mode .....         | 10 |
| 1.7. Turn the Benefits Row into a Flexbox Row .....                   | 11 |
| 1.8. Make Each Card Claim an Equal Share.....                         | 14 |
| 1.9. Copy Your Winning Rules into styles.css .....                    | 15 |
| 1.10. See the Mobile Problem .....                                    | 16 |
| 1.11. Add a Media Query to Stack on Phones .....                      | 17 |
| 1.12. Confirm the Full Layout Still Works.....                        | 18 |
| 1.13. Bonus: Add a Tablet Breakpoint for a 2-Across Layout.....       | 19 |
| 1.14. Review .....                                                    | 20 |

# Evergreen Insurance Quote Page Responsive Layout with Flexbox

**LAB 1**

In this lab, you will open the Evergreen Insurance Quote Page in Visual Studio Code and add a new **"Why Evergreen"** section that holds three benefit cards. In the starter, those cards stack vertically because HTML `<div>` elements default to block layout — each one takes its own row. Your job is to turn them into a horizontal row on desktop and stack them cleanly again on phones.

You are not designing a new component this week. You are taking an **existing** stack of cards and applying two small CSS rules so the same HTML lays itself out differently at different screen widths. This is exactly the work a front-end engineer does when the design team says "make this responsive."

## 1.1. A Note About Keyboard Shortcuts

Your lab VM runs inside your browser. Some keyboard shortcuts (like `Ctrl+Shift+P`, `Ctrl+S`, `Ctrl+F`) may not always reach the VM — sometimes your browser catches them first. If a shortcut doesn't do anything, that's not you doing something wrong; just use the mouse/menu alternative that appears next to every shortcut in this lab. Every step in this lab lists both.

## 1.2. Why You're Doing This Lab

Flexbox and media queries are the two highest-leverage concepts in modern layout. A page without them looks the same on every screen — which means it looks wrong on most of them. A page that uses them adapts: three columns on a laptop, one column on a phone, no JavaScript involved.

This lab shows you that exact transformation on a real (small) page. By the end you will have turned three stacked cards into a clean three-across row on desktop and a tidy vertical stack on phones — using a single Flexbox rule and a single media query. You'll finish with two small changes to `styles.css`:

- A Flexbox rule on the `.benefit-list` container (plus a `flex: 1` hint on each card)

- A single `@media (max-width: 640px)` block that flips the row back to a column on narrow screens

That is the entire payload of this lab. Two rules. Huge visual effect.

#### Tip

The pedagogical win in this lab is **watching the browser respond to rules you change live in DevTools**. Do not just type the final answer into `styles.css` and reload. Play with the values in DevTools first, see what happens, then copy what works into the stylesheet.

## 1.3. What's Already Set Up on Your Lab VM

Your lab machine runs Ubuntu Linux and comes pre-installed with everything you need for this lab. You do not need to install anything.

- **Visual Studio Code** — the code editor you used in earlier weeks.
- **Live Server extension** — already installed.
- **Git** — this week's lab folder is already initialized.
- **Firefox** and **Chrome** — either browser can be used. This lab uses Chrome DevTools' Device Mode and Flexbox overlay, so Chrome is recommended.

## 1.4. Open the Starter in VS Code

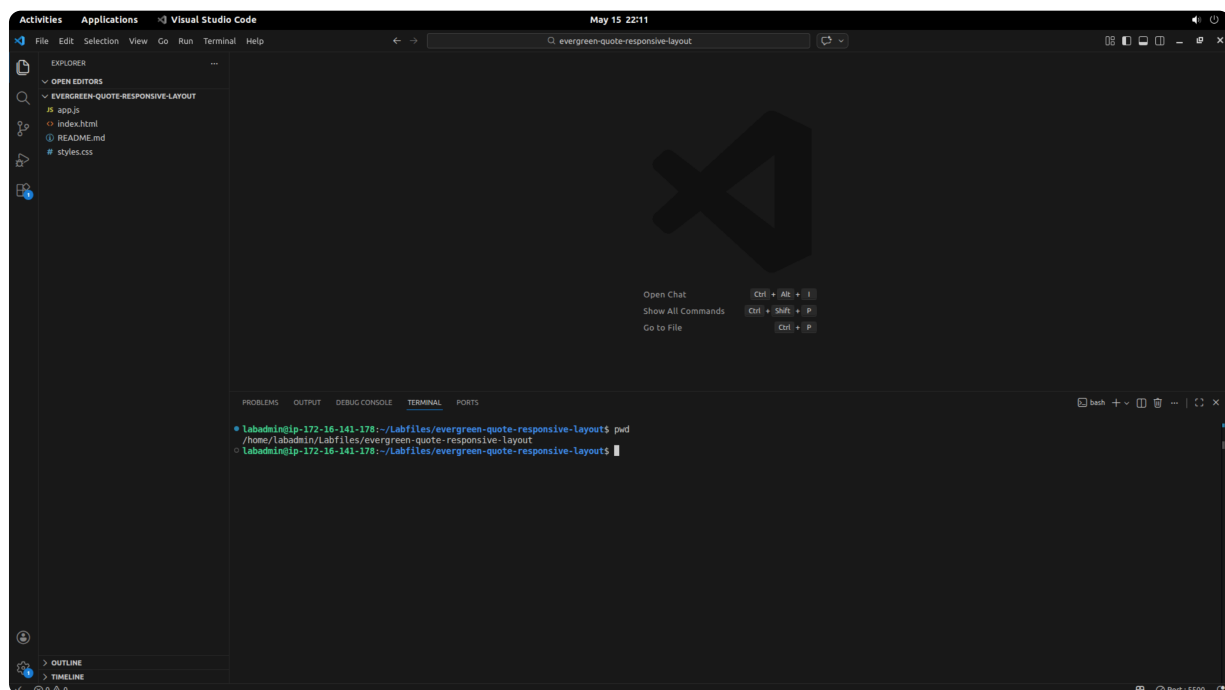
1. Launch VS Code from the lab VM's applications menu (or the desktop shortcut).
2. In VS Code, choose **File > Open Folder**, navigate to `Labfiles/evergreen-quote-responsive-layout` in your home folder, and click **Open**.
3. Open the VS Code integrated terminal. Two ways:
  - **From the menu:** Click **Terminal > New Terminal**.
  - **With a keyboard shortcut:** Press `Ctrl+``.
4. Confirm you are in the lab folder:

```
pwd
```

You should see a path ending in `Labfiles/evergreen-quote-responsive-layout`.

The project opens with four files:

- `index.html` — the Evergreen page structure, now including a new `<section class="benefits">` block with three benefit cards
- `styles.css` — the previous-week stylesheet (CSS variables, corporate blue rebrand) plus placeholder styles for the new benefits section
- `app.js` — unchanged from a previous week, handles the quote form
- `README.md` — containing instructions and information about the lab



## 1.5. Launch the Page with Live Server

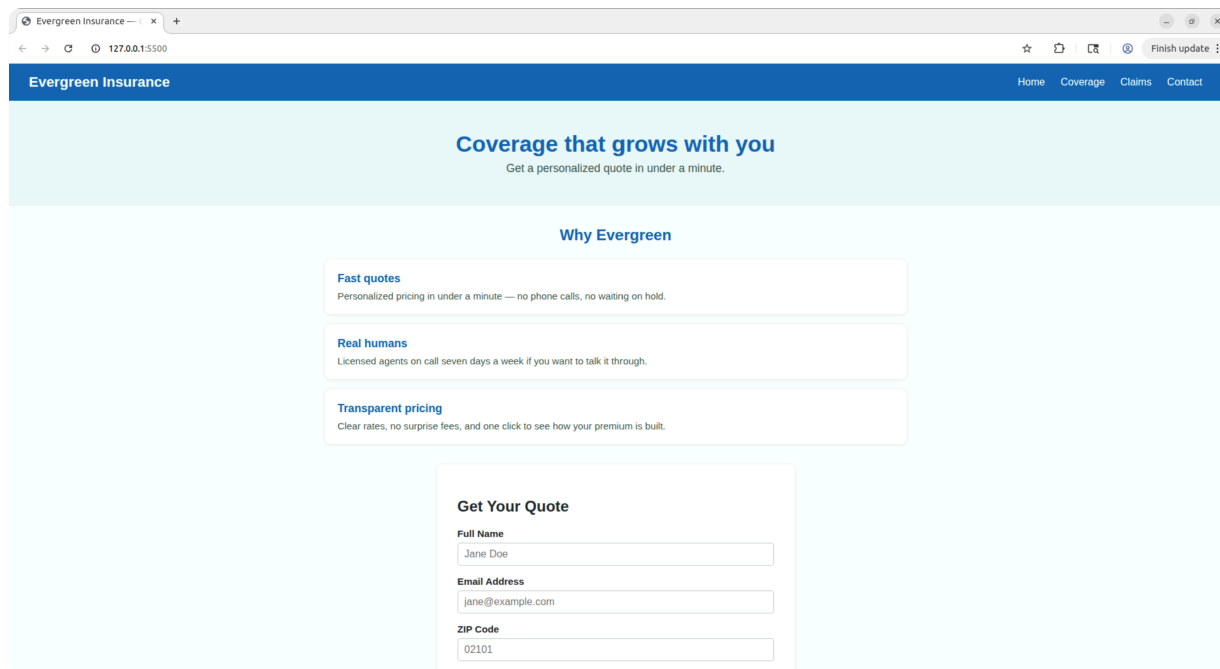
1. In the VS Code status bar at the bottom of the window, look for the **Go Live** button. Click it.

 **Note**



If you do not see **Go Live**, install the **Live Server** extension: **View > Extensions**, search for "Live Server" by Ritwick Dey, click **Install**, then try **Go Live** again.

2. The page opens in your browser (Firefox or Chrome — either works) at <http://127.0.0.1:5500/>. Keep the VS Code window and the browser window side by side if your screen allows.

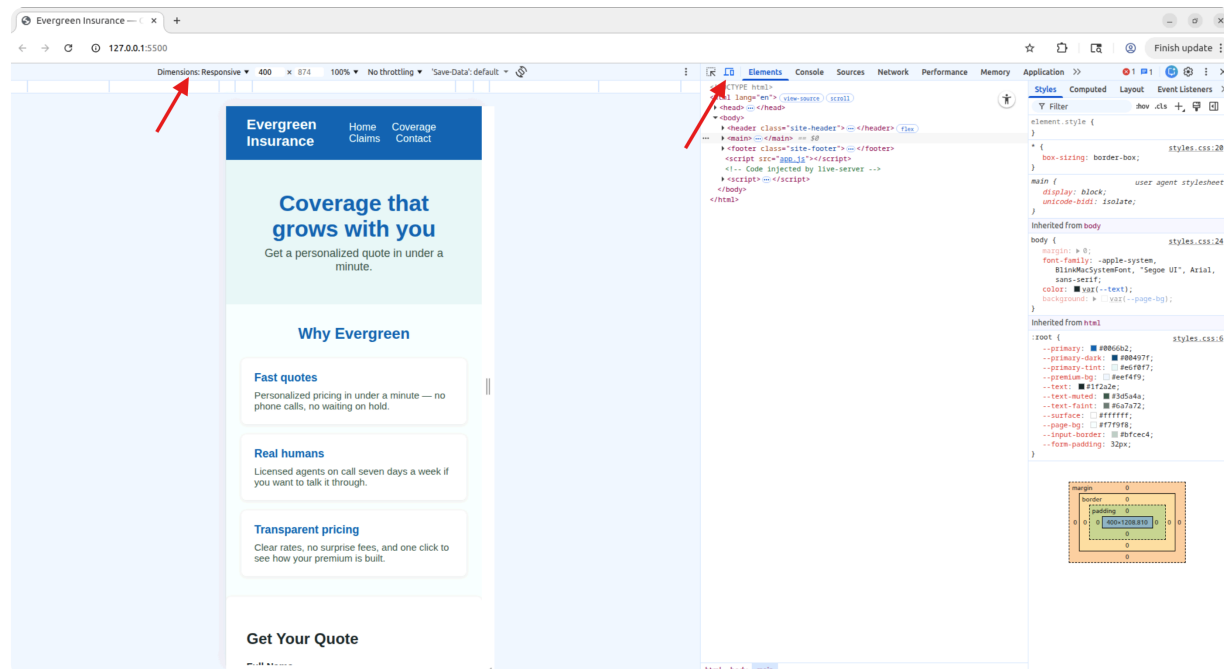


1. Notice how the three benefit cards — **Fast quotes**, **Real humans**, and **Transparent pricing** — are stacked vertically even at full desktop width. That is the block-layout default that HTML gives you for free. It is not what a designer would ask for. Our goal is to spread them across the row.

**Why this matters:** When your engineers say "the cards aren't laying out right," they almost always mean exactly this — block-by-default HTML producing a tall, awkward stack where the design calls for a row. Recognizing the symptom on sight (and knowing it is a one-line CSS fix, not an HTML rewrite) means you can scope a "make this responsive" ticket in a planning meeting in seconds instead of deferring to engineering.

## 1.6. Observe the "Before" State in DevTools Device Mode

1. Open Chrome DevTools. Two ways:
  - **With the mouse:** Right-click anywhere on the page and choose **Inspect**.
  - **From the menu:** Click the three-dot menu in the top-right of Chrome, then **More Tools > Developer Tools**.
2. Click the **Toggle device toolbar** icon in the top-left corner of the DevTools panel — it looks like a small phone-next-to-tablet icon. The page reloads inside a virtual viewport you can resize.



1. From the **Dimensions** dropdown at the top of the device toolbar, pick **iPhone 14 Pro Max** (or any phone preset). The page snaps to mobile width.
2. Scroll down to the benefits section. At phone width, the vertical stack actually looks OK — each card takes the full width of the viewport, which is fine for a phone. So the "problem" is only at desktop width: the same stack looks weirdly tall and wastes horizontal space.
3. Switch the Dimensions dropdown back to **Responsive** and drag the viewport wider than 900 pixels. The three cards are still stacked. **This** is the problem we are about to fix.

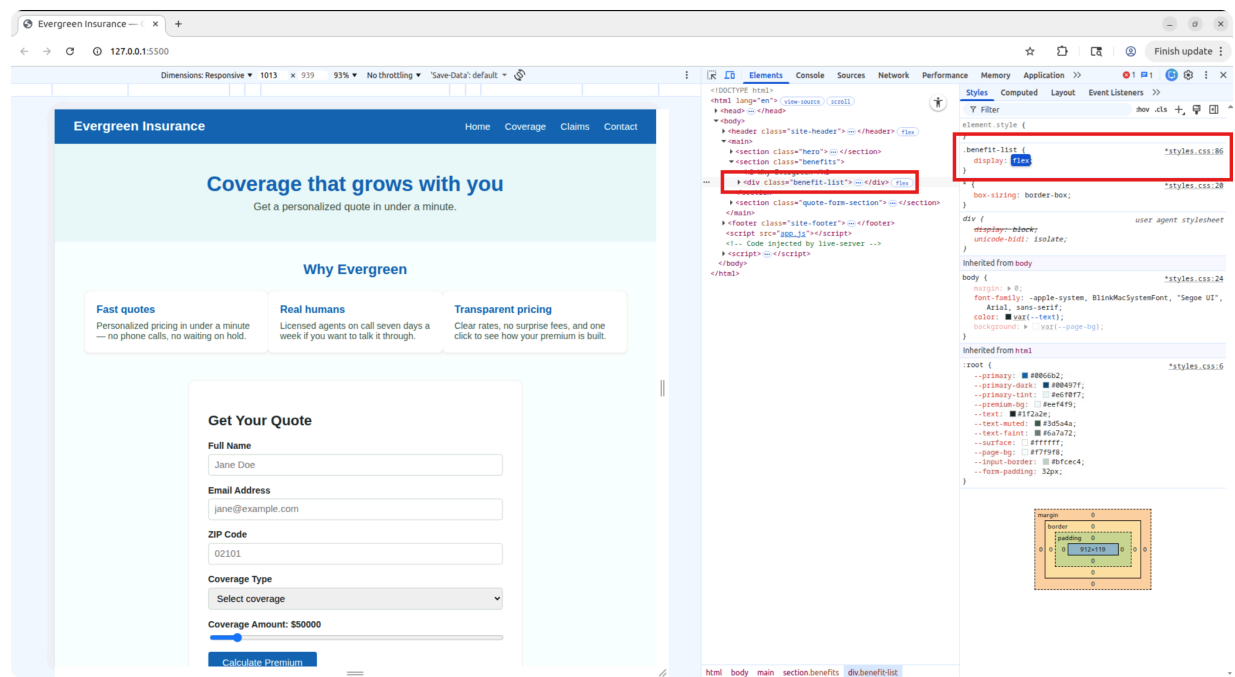
**Why this matters:** Device Mode is how a front-end engineer checks responsiveness without owning a drawer full of phones. Every change you ship to a public website is going to be viewed on screens between roughly 320 and 2560 pixels wide, and Device Mode is the cheapest, fastest way to see all of those at once. Expect your engineers to have this open on a second monitor whenever they are touching layout.

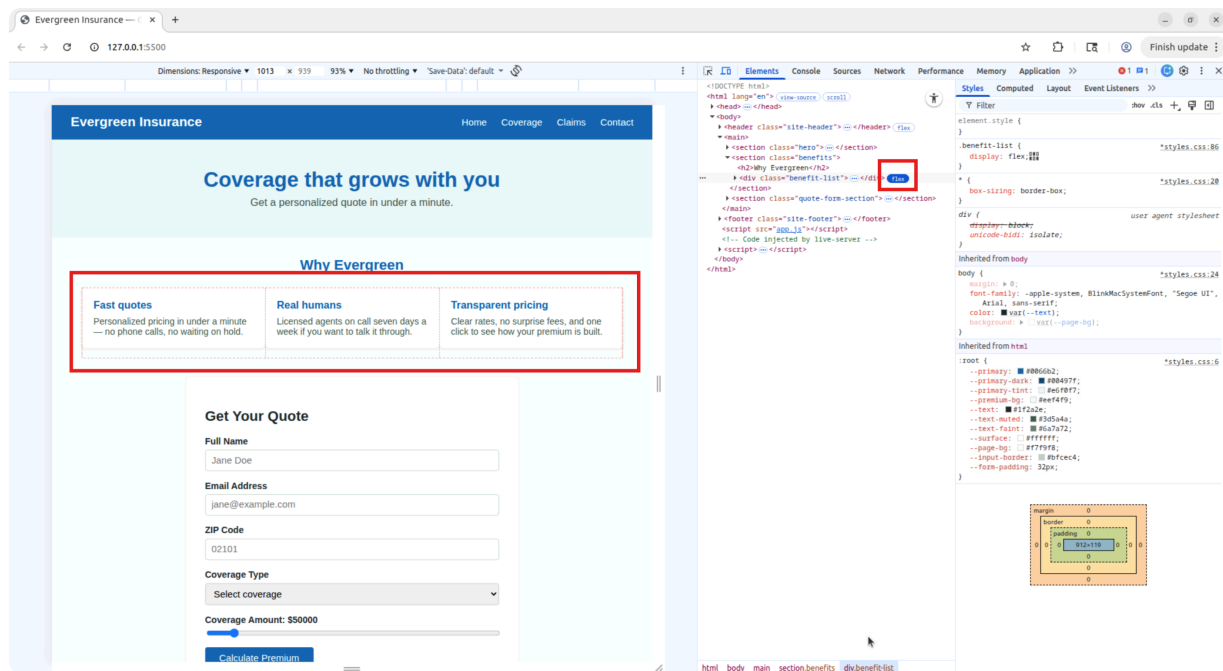
## 1.7. Turn the Benefits Row into a Flexbox Row

1. In DevTools, click on the benefits container — the `<div class="benefit-list">` element. You can use the pick-an-element tool (the cursor icon in the top-left of DevTools) and click the area around the three cards.
2. In the **Styles** pane on the right, find the `.benefit-list` rule that comes from `styles.css`. You are going to add three declarations to it interactively, then copy them into the stylesheet.
3. First, click inside the rule body (just below the opening `{`) and type:

```
display: flex;
```

The moment you press **Enter** or click out of the field, the three benefit cards snap into a horizontal row. That is the single most important visual of the week.





1. Next, click at the end of the `display: flex;` line in the Styles pane and press **Enter** to add a new line. Type:

```
gap: 24px;
```

The cards now have consistent spacing between them. Notice that without `gap`, the three cards would be touching edge-to-edge — `gap` is the clean modern way to separate flex children, replacing the old trick of adding margins to each one.

2. Finally, add:

```
justify-content: space-between;
```

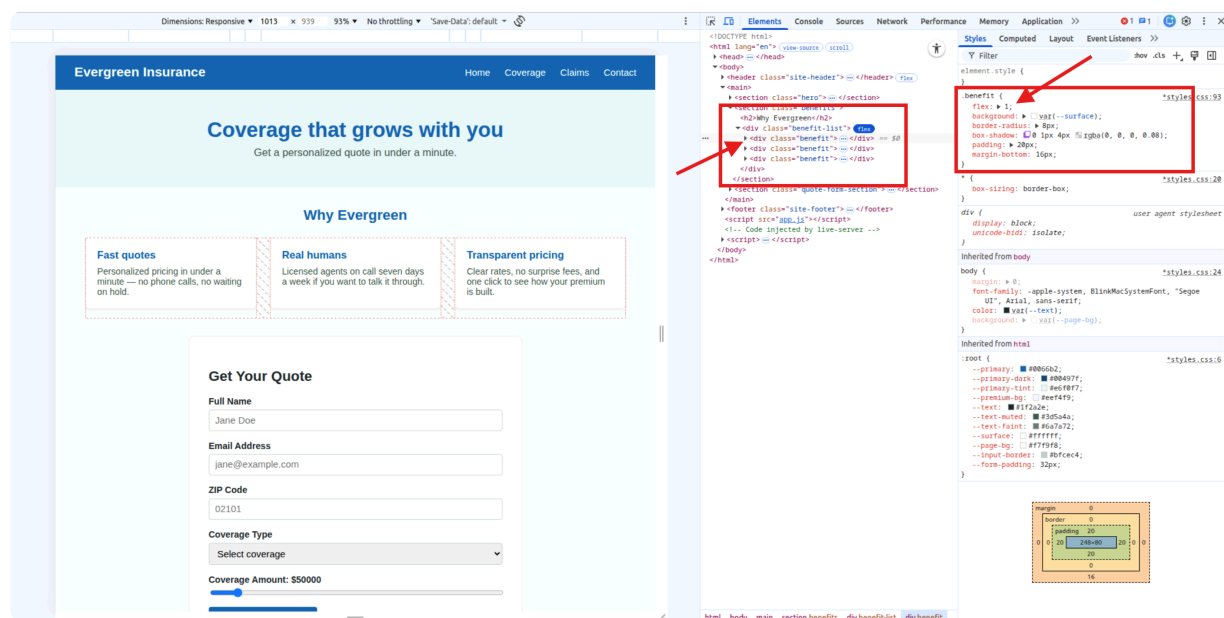
## 1.8. Make Each Card Claim an Equal Share

With `display: flex` alone, each card takes only the width it needs, leaving empty space at the end. We want the three cards to share the row equally.

1. Click on the first `.benefit` card in the Elements panel. In the Styles pane, find the `.benefit` rule. Add one declaration:

```
flex: 1;
```

All three cards now expand to fill the row in equal shares. `flex: 1` is shorthand that tells each card "grow to fill available space alongside your siblings." You do not need to memorize the full shorthand; recognize that one line makes equal columns out of flex children.



## 1.9. Copy Your Winning Rules into `styles.css`

DevTools changes are temporary — refresh the page and they vanish. Now that you have seen the values work, reflect them back into the stylesheet so the change persists.

1. Open `styles.css` in VS Code. Find the `.benefit-list` rule (around line 86):

```
.benefit-list {  
  /* TODO in lab: turn this into a Flexbox row with gap and  
  justify-content */  
}
```

2. Replace the comment with the three declarations you tested:

```
.benefit-list {  
  display: flex;  
  justify-content: space-between;  
  gap: 24px;  
}
```

3. Find the `.benefit` rule just below it and add `flex: 1;` as its first declaration:

```
.benefit {  
  flex: 1;  
  background: var(--surface);  
  ...  
}
```

4. Save the file. Two ways:

- **From the menu:** Click **File > Save**.
- **With a keyboard shortcut:** Press `Ctrl+S`.

Live Server reloads the browser. The cards should still be in a row at desktop width — now permanently.

### Note

If the browser still shows the old stacked layout after saving, click the browser window to focus it and press `Ctrl+Shift+R` for a hard refresh (or use the browser's **Reload** menu item). Sometimes Live Server holds a stale copy.

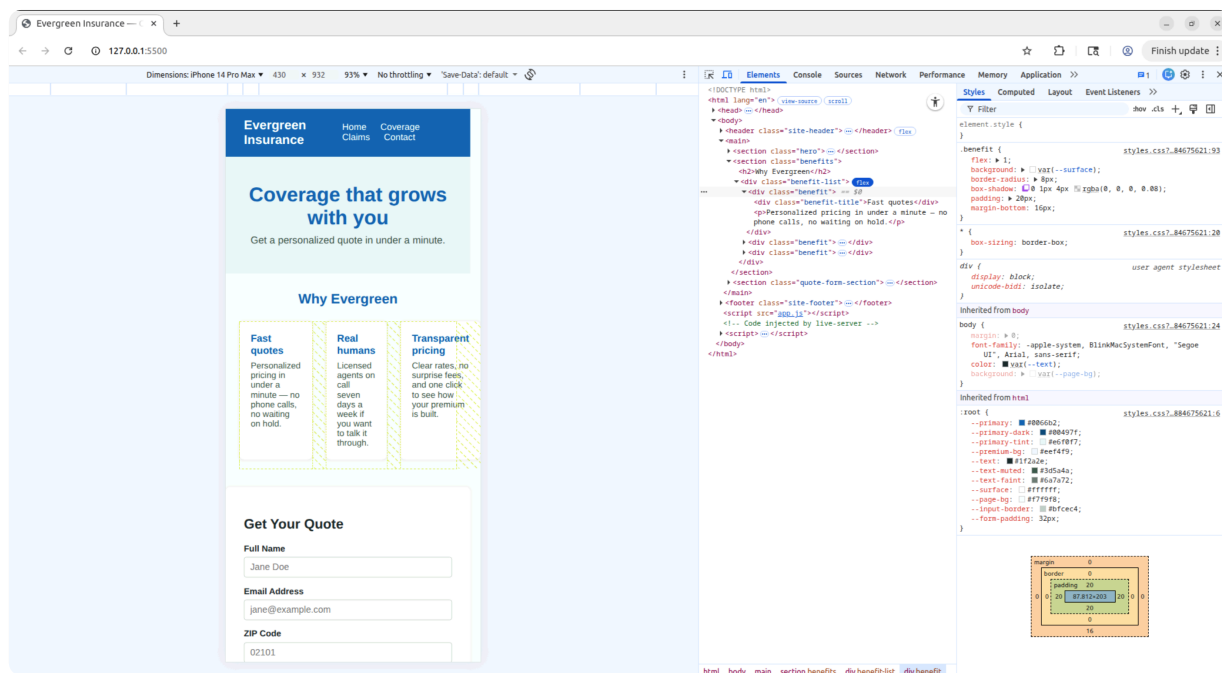
**Why this matters:** Prototyping in DevTools and then promoting the winning values into the stylesheet is the standard inner loop for CSS work. The DevTools sandbox lets an engineer try ten things in a minute; the stylesheet is where exactly one of those ten gets to live. Recognize this rhythm on a screen-share and you will know the work is going well.

## 1.10. See the Mobile Problem

1. Switch DevTools Device Mode to **iPhone 14 Pro Max** again. The three benefit cards are now **squished** — each one sits in roughly a third of a phone screen, which is far too narrow for a paragraph of text to read comfortably.

This is what responsive design is **for**. A rule that looks great on desktop ("three columns!") can look actively bad on a phone. Media queries let you write layout rules that only apply at certain screen widths.





## 1.11. Add a Media Query to Stack on Phones

1. Back in `styles.css`, add the following block **just after** the `.benefit` rule (anywhere in the benefits section is fine, but this is where it most logically lives):

```
/* Responsive: on narrow screens, stack the benefit cards
vertically */
@media (max-width: 640px) {
  .benefit-list {
    flex-direction: column;
  }
}
```

2. Save the file.
3. Switch to **Responsive** mode in DevTools and drag the viewport narrower than 640 pixels. The moment the width crosses 640, the cards snap into a vertical stack again. That is responsive design, in the most honest possible form.

4. Drag the viewport wider, past 640 pixels. The moment the width crosses 640, the cards snap back into a row. Drag narrower again — they stack. That is responsive design, in the most honest possible form.

#### Tip

640 is a common "phone vs tablet" threshold, but not a magical number. A good follow-up conversation with an engineering partner is to ask what breakpoints your team uses on the property you care about.

**Why this matters:** A **breakpoint** is the screen width at which your layout changes. Every responsive website has a small handful of them, picked by the design system team. When you hear "we broke the layout at the tablet breakpoint" in a bug triage, you now know it means a specific media query produced a specific visible glitch at a specific width — and you know where in the stylesheet to point if you want to see it for yourself.

## 1.12. Confirm the Full Layout Still Works

1. Drag through a few widths to check the whole page:
  - At ~1200 pixels: three-across benefit cards, wide quote form in center
  - At ~800 pixels: three-across benefit cards, same quote form
  - At ~640 pixels: benefit cards stack (media query fires), quote form still readable
  - At ~400 pixels: cards stacked, form fills most of the width
2. Your changes should not affect the header, hero, form, or footer at any width. If any of those look wrong, open the Styles pane on the misbehaving element and look for stray rules you may have edited by accident.

## 1.13. Bonus: Add a Tablet Breakpoint for a 2-Across Layout

So far the benefits section has exactly two states: three columns on anything wider than 640 pixels, one column on anything narrower. That works, but it ignores the middle of the spectrum — tablets and small laptops in the ~700–900 pixel range, where three columns are slightly cramped but one column wastes space. If you want five extra minutes of practice, add a second media query that gives tablet-sized screens a 2-across layout. This stacks two breakpoints on top of each other and shows you the cascade of responsive rules in action.

1. Open `styles.css` in VS Code and scroll to the benefits section where you added your `@media (max-width: 640px)` block.
2. **Just above** that existing block, add a second media query that targets the tablet range:

```
/* Responsive: at tablet widths, drop from 3 columns to 2 */  
@media (max-width: 900px) {  
  .benefit {  
    flex: 1 1 calc(50% - 24px);  
  }  
}
```

The `flex: 1 1 calc(50% - 24px)` shorthand tells each card "your ideal width is half the row minus the 24px gap, but you may grow or shrink from there." Combined with the `flex-wrap: wrap` you will add next, two cards fit per row and the third wraps to a second row.

3. The wrap only works if the flex container is allowed to wrap its children. Find your `.benefit-list` rule and add `flex-wrap: wrap`:

```
.benefit-list {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: space-between;  
  gap: 24px;  
}
```

4. Save the file ( `Ctrl+S` , or **File > Save**). Live Server reloads the browser automatically.
5. Switch DevTools to **Responsive** mode and drag the viewport across the full width range. You should now see three layout states instead of two:
  - Above ~900 pixels: three cards in a row (your original Flexbox rule).
  - Between ~640 and ~900 pixels: two cards on top, one card wrapped to a new row.
  - Below ~640 pixels: all three cards stacked vertically (your original mobile media query).
6. Watch the transitions carefully as you drag. At each breakpoint the layout snaps to its new shape **because two media queries are competing**, and the narrower one wins when both match. That cascade is the whole mental model of responsive CSS.

**Why this matters:** Real production layouts rarely have just one breakpoint — most design systems define three or four (phone, tablet, desktop, wide desktop). The pattern you just practiced — stack media queries from widest to narrowest, let each override the last — is exactly how engineers build a layout that looks intentional at **every** width, not just the two you happened to test. When an engineer says "the tablet breakpoint is off," you now know to look for the middle media query in the cascade.

## 1.14. Review

In this lab, you:

- Opened the Evergreen quote page and observed the default block-stacked benefit cards.
- Used Chrome DevTools Device Mode to see how the same HTML looks at desktop and phone widths.
- Added `display: flex` , `gap` , and `justify-content` live in DevTools to turn the stack into a row.
- Added `flex: 1` to each card so the three share the row equally.
- Promoted the winning DevTools values into `styles.css` .

- Added a single `@media (max-width: 640px)` block that flips the row back to a column on phones.
- Reviewed the full change as a Git diff and committed it with a clear message.

You changed roughly six lines of CSS — but you made a section of the page properly responsive. That is the leverage Flexbox and media queries give a modern layout.

The next time an engineering team says "we need to make this responsive," you know what they mean, roughly what the code looks like, and which DevTools features they are going to use to debug it. You also know why the word **breakpoint** matters — it is where layout changes.

Keep DevTools open on every web page you touch going forward. The more you use the Device Mode toggle and the Flexbox overlay, the more fluent your design-review instincts become.